

CALY

Proposal

App name

Caly

The problem

People who want to monitor their calorie intake often stop logging food because traditional calorie trackers make every entry feel like a form, while Caly lets them record food as quickly as writing a normal note.

Who is this for

Caly is for students, young adults, and beginners who want to become more aware of their daily calorie intake without using a complicated fitness or nutrition app.

Today, they may rely on memory, type meals into Apple Notes or another notes app, search calorie values separately, or use calorie trackers that require food search, serving-size forms, and several confirmation steps.

Caly keeps the familiar note-taking behavior. Breakfast, Lunch, and Dinner each have an Add food action; the user taps the meal they want, types one food, the calorie value appears on the right, and the daily total updates at the bottom.

```
NOTES
Breakfast
1 cup rice          200 kcal
2 boiled eggs       150 kcal
Add food...
Lunch
Add food...
Dinner
Add food...
Total               350 kcal
```

Core features (MVP)

The main interaction is organized by meal section. Breakfast, Lunch, and Dinner each have their own Add food action. The user taps Add food under the meal they want, Caly creates the entry in that section, the user types the food, the app sends that food text to the FastAPI backend, FastAPI sends it to the Gemini API, Gemini interprets the food and returns a calorie count, Caly shows the calories beside the line, and the daily total is recalculated. Every new food entry repeats the same cycle. The meal section is chosen by the user in Caly; Gemini does not decide whether an entry is breakfast, lunch, or dinner.

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest estimate
1	Notes-style meal-section food entry	keep	StatefulWidget, MealSection, AddFoodRow, TextField, TextEditingController, FocusNode, Row, Expanded, http, FoodEntry model	8 hours
2	Automatic calorie total	keep	setState, List<FoodEntry>, Text, calculated getter/method	2 hours

3	Food entry correction	keep	showModalBottomSheet, TextField, DropdownButton/menu, FilledButton	3 hours
4	Daily food journal and history	keep	ListView.builder, StreamBuilder/FutureBuilder, Navigator, cloud_firestore, DailyNote model	5 hours
5	User account and cloud saving	keep	firebase_auth, cloud_firestore, TextField, Form, loading/error states	5 hours

Total estimated implementation time is about 23 hours, plus integration testing and visual polish. The note screen remains the priority; the backend exists to support that one fast interaction rather than to add a separate assistant or chatbot experience.

Stretch goals

1. Weekly insights such as average calories and frequently eaten foods.
2. Voice input for adding a food without typing.
3. Protein, carbohydrate, and fat information.
4. Saved or frequently eaten foods for faster reuse.

How my app saves data

The question from page 1: No. Two different people should not see the same food journal. Each signed-in user has a separate journal. If the same person signs in on another device, that person should be able to retrieve the same saved journal.

Roughly how many records: A realistic week contains about 7 DailyNote records and roughly 70–100 FoodEntry objects, plus one small user/settings record.

My choice: Firebase Authentication + Cloud Firestore.

Why this one and not the others: Caly already needs a user account and cloud saving, so Firebase fits the actual feature set better than shared_preferences, which would tie the journal to one installation. Building my own database behind FastAPI would give more control but would also add database administration and authentication work. The tradeoff is that Firebase introduces network and configuration work, but it keeps account identity and journal persistence in one managed backend.

What I save, concretely:

- Firebase Authentication: uid and email for the signed-in user.
- Cloud Firestore users/{uid}: createdAt and basic profile information.
- Cloud Firestore users/{uid}/daily_notes/{yyyy-MM-dd}: date, title, totalCalories, createdAt, updatedAt, and entries.
- Each FoodEntry inside entries: id, originalText, foodName, quantity, unit, calories, mealCategory, isManuallyCorrected, and createdAt.
- Cloud Firestore users/{uid}/settings/main: dailyCalorieGoal and appearance.

Have I tried it yet? Not yet. By September 22, 2026, I will complete a one-hour Firebase spike: register one test account, save one DailyNote with three food entries, restart the app, sign in again, and confirm that the same note returns. I will also test a second account to confirm that it cannot read the first account's journal.

One thing I want to add that the course did not teach

The new part I want to add is the Gemini API for interpreting food input and returning its calorie count. This is a core feature because the intended Caly flow depends on the calorie result appearing from a normal typed food line.

- Flutter package: http, used only to call my FastAPI endpoint.

- Backend: FastAPI with the Google GenAI Python SDK. The Gemini API key is created/managed through Google AI Studio and is stored only on the FastAPI side.
- Program flow: Flutter sends one food line to POST /api/v1/foods/interpret → FastAPI validates it → Gemini returns the interpreted food and calorie count → FastAPI returns a small JSON response → Flutter updates that FoodEntry and recalculates the sum of all calorie values.
- Where it runs: Flutter runs on mobile and web; FastAPI runs as a separate Python service, so the browser version can call the same endpoint.
- Web fallback: if the backend is unavailable, the app still opens and every screen remains reachable. A failed row can be retried or corrected manually; the finals video will show the live food-to-calorie flow.
- Priority: Core feature. The first backend spike will connect one TextField entry to FastAPI and return one calorie count before secondary screens are polished.

How my project runs when someone else opens it

- I am keeping the device_preview wrapper: Yes. Caly is designed for phone proportions, and device_preview keeps the browser demo close to the intended mobile layout.
- My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable: Not yet. The target is full browser navigation with Firebase configured for Web and the FastAPI endpoint available to the demo origin.
- Anything that needs real hardware degrades to sample data instead of crashing: Not applicable to the MVP. Caly does not require camera, GPS, microphone, or another hardware-only feature.
- Public repository and secrets: the Gemini API key will live in a git-ignored FastAPI .env file during development and in the backend deployment environment in production. It will never be hard-coded in Flutter or committed to Git. Firebase access will be protected by Authentication and Firestore security rules. Demo account data will be fictional.

Data the app remembers

Thing	Fields	Where it is saved
User account	uid, email, createdAt	Firebase Authentication + Firestore users/{uid}
Daily note	date, title, totalCalories, createdAt, updatedAt, entries	Firestore users/{uid}/daily_notes/{yyyy-MM-dd}
Food entry	id, originalText, foodName, quantity, unit, calories, mealCategory, isManuallyCorrected, createdAt	Inside the entries field of its DailyNote
App settings	dailyCalorieGoal, appearance	Firestore users/{uid}/settings/main

Screens

1. Sign In / Registration — the user creates an account or signs in, then enters the journal.
2. Today's Food Note — the main note screen. Breakfast, Lunch, and Dinner each have an Add food action. Tapping Add food creates an editable entry in that section; after the user types the food, the calorie count appears on the right and the running total updates.
3. Food Correction — opened by tapping a calorie value; the user can correct the interpreted food, quantity, serving, or calorie value.
4. Food History — shows saved daily notes and lets the user reopen a previous day.

5. Settings — daily calorie goal, appearance, account information, clear-journal action, and sign out.

Risks

The risk I named last time: Food interpretation and calorie accuracy is still the biggest product risk. The input can be vague, such as “rice,” “coffee,” or “chicken,” and the returned calorie count depends on how the food and serving are interpreted.

First step to reduce it: By September 22, 2026, I will test POST /api/v1/foods/interpret with at least 20 realistic food lines, including clear and vague entries, and record whether the returned food, quantity, unit, and calories are usable. The correction screen remains the user’s fallback when a result is wrong.

A new risk I did not see before: Firebase, FastAPI, and the external food request all add asynchronous work to an interface that is supposed to feel like a simple note.

First step to reduce it: By September 23, 2026, I will build one complete Today-screen path: tap Add food under Lunch → type one food → receive calories → show the value → update the total → save the entry with mealCategory set to Lunch. I will test success, retry, and manual correction before building additional polish.

What changed, and why

Section	Prelim said	Now says	Why it changed
Smart food note	Gemini interpreted the food, while a nutrition database or FastAPI nutrition service was planned to calculate calories.	The Gemini API interprets the food and returns the calorie count; FastAPI is the secure application layer between Flutter and Gemini.	This removes a second calorie source and gives the main input flow one clear request/response path.
FastAPI	Mentioned as a possible nutrition service.	Required backend endpoint for food input; it keeps the Gemini API key off the Flutter client and validates the response.	The app now has a concrete backend responsibility instead of an optional service.
Firebase	Account and cloud saving were planned generally.	Firebase Authentication and Cloud Firestore are named with exact paths and saved fields.	The initial idea already needed accounts and cloud persistence; the revision makes the storage plan buildable.
Screens	Six screens including Simple Insights.	Five core screens: Sign In / Registration, Today, Food Correction, Food History, and Settings. Insights is stretch.	After building multi-screen Flutter apps, five core screens is a more realistic term scope while keeping the complete journal journey.
Food row display	The initial plan included confidence/estimate information around recognition.	The normal display is only food text and its calorie value. Processing stays visually quiet, and errors only expose a simple retry/correction action.	The note should look like a note, not expose the implementation details behind every calorie result.
Saved food-entry fields	Confidence level and estimation flags were included.	Only the input, interpreted food data, calories, manual-correction flag, and timestamps are saved.	Those technical fields are not needed for the user-facing journal or the revised persistence plan.
Meal organization	The initial proposal organized daily entries under meal headings such as breakfast, lunch, dinner, or snacks.	Breakfast, Lunch, and Dinner each have their own Add food action. The selected section supplies mealCategory before the food text is sent for calorie interpretation.	This keeps the interaction predictable and keeps the calorie service focused only on the food text and calorie result.